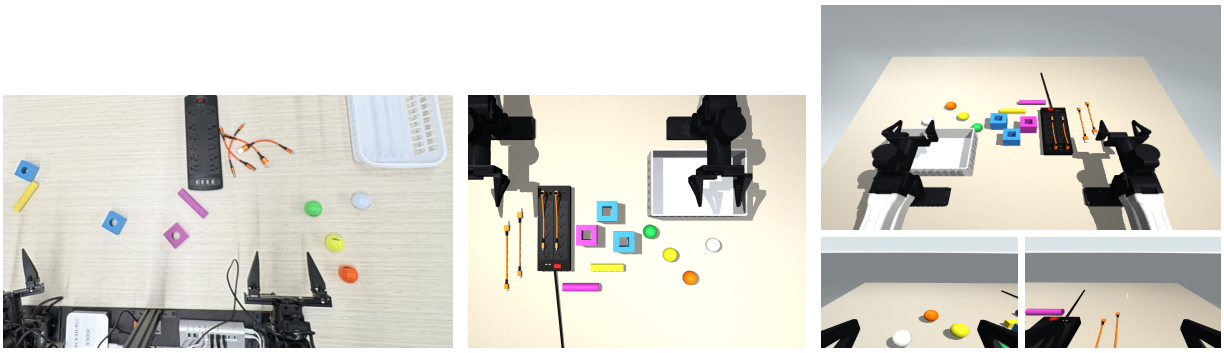
Supervising a Hosted Vision-Language-Action Policy Through a Rule-Bound, Long-Horizon Relay: Harness Design, Classifiers and Simulation

Daniel Carrera David Carrera Kaushik Kanna Muhammad Mukthar
Darsh Patel

Physical AI Hacks (MakerMods × Dream Labs), 7–8 November 2026
Authors listed alphabetically by surname. Preprint, 9 November 2026
Code: github.com/darshp848/hill-climb-harness, task_simulation, follower_code



(a) Station C from above (b) Simulated table, top-down (c) Simulated policy cameras

Figure 1: The relay and its simulation. (a) The physical table on 7 Nov, cropped to the table: the grippers of two MakerMods Metal arms at the bottom edge, a power strip with orange USB cables, foam cubes and sticks, golf balls and a white bin; the white ball is illegal. (b) Our MuJoCo model of the same table after its 8 Nov rebuild: the strip's real dimensions and ports, holed cubes, sticks, a slotted basket and the station's ball colours. (c) What the policy sees in simulation: overhead, left wrist and right wrist at the station's 640×360.

Abstract

We describe the system our team built in 36 hours to run a hosted generalist vision-language-action (VLA) policy, Dream Labs' Dream-1, through a timed relay on two brakeless MakerMods Metal arms: unplug USB cables from a power strip, stack a tower over 4 in, and bin legal golf balls while never binning the illegal one. The policy is a black box behind an HTTP station agent, so the only inputs a team controls are the words it sends and when it stops, homes or retries. The rubric makes supervisor errors fatal: touching a later stage's object before the current pass bar is a disqualification. Our harness runs the relay as a stage state machine, composes every instruction from tested templates, and decides progress with a layered set of classifiers: a vision-language judge for stage completion, a local colour-and-shape detector, a 10 Hz illegal-ball guard that stops the arms on its own, a MobileNetV3 cable-end classifier, gripper-width rules, and learned per-joint fault detectors. We built two simulators to test it, a virtual-clock stress simulator with 37 fault scenarios and a MuJoCo table driven by the live Dream-1 API, which we later rebuilt to match the station's objects. On the station, the median policy round trip was 1.12 s and 67% of 1,650 chunks ran stop-and-wait. Retuning the detector on station frames raised held-out ball recall from 20/27 to 26/27 at 100% precision; retraining the cable-end classifier on 478 wrist crops raised USB-A recall from 0.68 to 0.84. In 185 simulated matches, a fast judge without the local guard lost 28 matches to the illegal ball and with it lost none. A study of 101 station rollouts found that naming the goal, not the motion, and adding a scripted pull made unplugging work: 8 of 12 harness pulls kept the plug. The team's fine-tune was worse than the base model, and we trace this to its

Table 1: Rubric v4 of the 1v1 Hill Climb Relay (abridged). Pass bars gate the next stage.

Stage	Pass bar	Points
1. Unplug USB cables	≥ 1 cable out of the strip	+10 first, +5 each further; +20 per loose cable plugged in (max 4)
2. Stack	tower > 4 in, free-standing 3 s after release	+20; +3 per full inch; +10 taller tower (head-to-head)
3. Balls	none	+5 per legal ball in bin; illegal ball in bin = match lost
Integrations	shown to referee before the match	+10 LiveKit, +5 Dimensional
DQ: touching a later stage's object before the current pass bar; any human input after "go".		

Quest demonstrations, whose commands sit 3.7 to 5.4 cm above the measured tool because of gravity sag. The team placed 5th of 8. We state throughout which numbers are simulated and which are real.

1 Introduction

Generalist VLA policies [1–3] map camera images and a sentence to a chunk of robot actions. They are increasingly served as an API: a station agent on the robot computer runs the control loop, and the model runs in the provider's cloud. A team using such a service cannot change the weights at run time or put a controller beneath them. It can choose words, judge progress, stop the arms, and collect demonstrations for a fine-tune.

The Physical AI Hacks "Hill Climb" relay made those choices decisive. A match is five minutes with no human input after "go". Points come from cables unplugged, tower height and golf balls binned (Table 1). Two rules turn ordinary errors into losses. Stages are gated, so touching a block before a cable is out, or a ball before the tower stands, disqualifies the team. And an illegal ball lies among the legal ones; binning it loses the match. A supervisor that declares "done" too early, or lets the policy wander toward the wrong object, scores zero no matter how capable the policy is.

Prior work on language supervisors [4–6] assumes a library of skills the planner can call. Here there is one monolithic bimanual skill, one prompt for both arms, a 1 s network round trip, and gates whose violation cannot be undone. The questions are therefore about supervision under latency and irreversibility: how to decide quickly and safely that a stage is done, how to stop a bad grasp in time, what to say to the policy, and how to test all of this before a station slot.

Contributions.

- **A supervisory harness for a hosted VLA** (Section 4) that never commands a motor. It runs the relay as a state machine with rubric-aware retry and cutoff rules, composes prompts from linted templates, adds two narrow harness skills (regrip and pull), and keeps the serving loop in asynchronous mode. We also write its decision rules as equations that can be tuned (Section 4.6).
- **A layered perception and monitoring stack** (Section 5): seven classifiers and estimators, from a multi-second vision-language judge to a 76 ms colour detector and sub-millisecond joint-fault networks, each placed by how fast and how trustworthy it is. We report how each was designed, trained and checked against station data.
- **Two simulators** (Section 6): a virtual-clock stress simulator that runs the real harness code against scripted faults, and a MuJoCo table driven by the live policy, rebuilt to match the station.
- **An evaluation** (Section 7) organised around six questions, from the serving loop to the fine-tune's failure, using station logs, 101 scored station rollouts, 185-match sweeps, a 25-configuration judge benchmark and MuJoCo matches.

The work spans three public repositories: the harness, the simulators and results, and the teleoperation tools. Our team placed 5th of 8 at the event. We have no logged per-stage success rate from complete harness matches on the station; Section 8 says what that leaves open.

2 Related work

VLA policies. RT-2 [1], OpenVLA [2] and π_0 [3] return action chunks from images and language. We treat such a model as a fixed, remote component and study the system around it.

Language supervisors with feedback. SayCan [4] scores skills by language relevance and affordance; Inner Monologue [5] feeds success detection and scene descriptions back into the plan; Code as Policies [6] writes the supervisor as a program. Our harness is closest to Inner Monologue, with two differences: the only actuator is a single prompt to one bimanual policy, and some supervisor mistakes are terminal by rule.

Success and failure detection. Vision-language models can act as success detectors [7] and explain failures [8]. We use one as a judge but keep the safety decision on a fast local detector, because no judge we benchmarked was reliable enough on the safety question alone (Section 7.3).

Teleoperation and data. Leader-follower arms [9, 10] and LeRobot datasets [11] are the usual route to fine-tuning data. Section 7.6 documents two collection paths on the same arm that disagreed by 3.7 to 5.4 cm in steady state.

Evaluation in simulation. SIMPLER [12] shows that simulated evaluation predicts real performance only when control and visual gaps are closed. That motivated rebuilding our MuJoCo table to the station's objects, and it is why we report stress-simulator scores as tests of harness logic only. For uncertainty we use bootstrap intervals as advocated by Agarwal et al. [13].

3 Task and platform

Station. Each Metal arm has six joints and a gripper, driven by Damiao motors over CAN at 1 Mbit/s from a NUC mini PC. There are no brakes. An overhead camera on a centre pole and one camera per wrist stream 640×360 frames (Figure 1a). On our table the strip is a black 12-outlet surge protector with a bank of USB ports at the robot end; the cables run USB-A to USB-C; the foam pieces are three 6 cm cubes with a through hole and two sticks; the legal balls are green, yellow and orange, and the white ball is illegal.

Control stack. We read the station's installed SDK (`dreamlabs` 1.0.37). The policy returns 48-row chunks played at 20 Hz (2.4 s per chunk). A command thread interpolates at 250 Hz, but the Metal bus thread reads joint state and sends MIT-mode setpoints at a fixed 30 Hz, so seven of every eight interpolated commands never reach a motor. The velocity feed-forward is low-pass filtered with a time constant of about 0.8 s. A client-side pacer requests chunks asynchronously, with the arm moving during inference, only while the median round trip is under 900 ms; otherwise it waits for each chunk.

Station API. The agent serves `/status`, `/start`, `/stop`, `/home`, `/reset` and `/prompt` on localhost. One prompt drives both arms; there is no per-arm command. A station add-on we wrote serves single 640-px frames (about 35 KB), gripper and joint readings (`/arms`), a bounded relative move used by the harness skills, and opt-in tuning hooks. Every add-on is off unless an environment variable enables it, and each refuses to load on any SDK other than the inspected version.

4 Harness design

Figure 2 shows the structure. Four principles shaped it.

1. **No motor driver in the harness.** All motion goes through the station agent, so safety caps and the control loop stay with the provider. Preflight refuses to start unless the station reports idle or error, both arms are alive and no frame is older than 2 s; faults, the clock and an operator stop all send `/stop`, which holds the arms. At the end of a run the bot plans send `/reset` to return the arms to the start pose.
2. **Keep safety off the slow path.** Anything that must stop the arms runs locally at frame rate and can act without the supervisor.

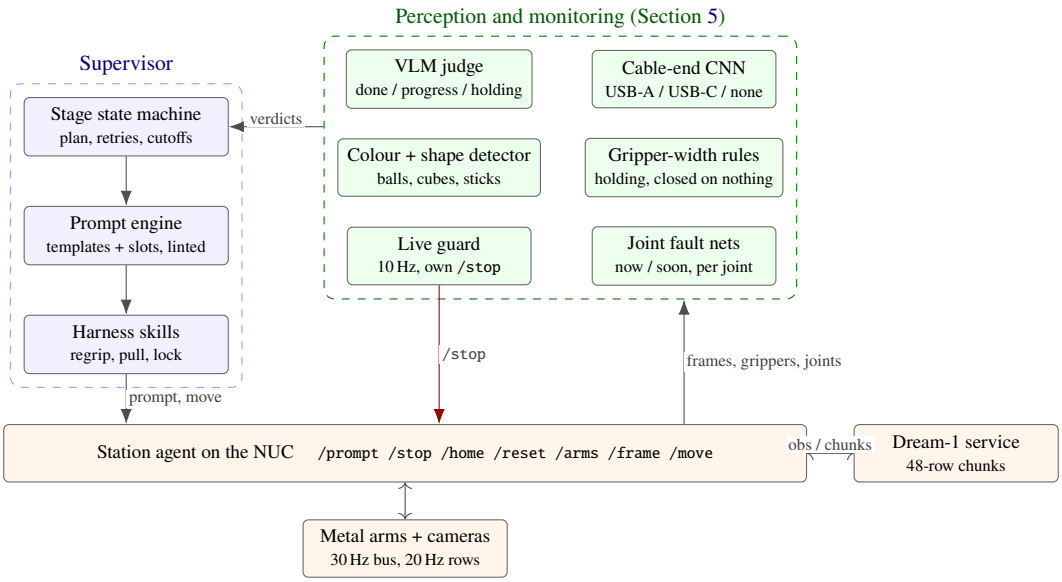


Figure 2: Harness overview. The supervisor talks only to the station agent: prompts, stop/home/reset, and a bounded relative move for its skills. Perception reads frames, gripper widths and joint state. The live guard has its own path to `/stop`, independent of the supervisor and the judge. Every motor command, speed cap and clamp stays inside Dream's control loop.

3. **Confirm from independent evidence.** A false "done" on a pass bar is a disqualification, so passes need two agreeing looks, and sensors can veto the image.
4. **No free text to the robot.** Every instruction comes from a template that worked on the station.

4.1 Stage state machine

A plan is a YAML list of stages with a prompt or template, a yes/no success question, a per-try timeout, a retry budget and an `on_fail` rule. The final order follows the team's operator decisions: unplug every seated cable, plug loose cables in, build the tower, then balls. Timeouts act at four levels: the 295 s match budget; per-stage cutoffs on the match clock (for example, no new plug-in try after 150 s), so a high-value but unreliable stage cannot starve later ones; a retry budget (two tries, three for stages worth 20 points or more); and a per-try timeout with two one-time extensions, 10 s when a "done" awaits its confirming look and 20 s when the gripper is on the target. Pass-bar stages are the exception to the retry budget. Moving on from one means touching the next stage's objects, so they retry until the clock runs out. Retries use `/prompt` on the running session; a fresh `/start` costs 2.9 s (median) before motion and sometimes 7–12 s of discarded first answers.

4.2 Prompt engine

Every instruction is `template[state, step, variant](slots)`. Slots name the object by colour, shape and position ("the orange golf ball on the right side of the table"). Templates are linted at load: a word cap, no negations, no arm named in an arm-agnostic template, no unfilled slot. A slot filler chooses among numbered options, from plan defaults, a single cheap judge call, or ground truth in simulation; a filler error falls back to the stage's default wording rather than ending the match. When the judge reports a stall, the filler re-fills slots with the judge's reason as a hint, and the text is resent only if it changed. Nothing is re-prompted while a gripper is closed and still on an object: on the station, 9 of 15 mid-grasp rewrites were sent while the arm held the plug, and 5 of those dropped it. For a fine-tuned checkpoint the plan switches to verbatim mode and sends the recorded task texts byte for byte.

The templates came from station trials with base Dream-1 (Section 7.5). Cable prompts carry a true safety sentence (the strip is unplugged, the cords are insulated low-voltage USB); without it the policy refuses cable work.

Table 2: Harness lines and what each contributed. All are in the hill-climb-harness repository.

Branch	Contribution
`main` (7 Nov)	state machine, judge verifier, two-look confirmation, colour detector, stress fixes
`start-pose-live-guard`	10 Hz live guard, start-pose gate, cable-end classifier, decision equations in shadow mode
`hybrid-harness`	colour/position prompts, judge-written prompt rewrites, safety sentence, light frames, gripper overrides
`final-harness`	merge of the three above with `-match` checks that refuse DQ-prone options
`cloud-sims`	async 2 Hz pacing, hedged policy calls, operator decisions as plan data, rebuilt sim table, cloud and cluster jobs, joint telemetry and fault nets
`hedged-calls`	hedged judge calls, OpenRouter judge benchmark
`fix/classifiers-8nov`	detector and cable-end classifier retuned on station frames
`prompt-engine`	template engine, relay state machine in the operator's order, sim-time clock
`claude/harness-stitch`	bot harness used on the station: grip assist, lock, hands-off mode, panel, station-side hedged calls
`claude/livekit-viewer`	view-only LiveKit observer, merged to `main` through PR #9
`relay-on-stitch`	prompt engine and relay order on top of the bot harness (PR #8, open)

4.3 Harness skills

Two unplug failures resisted wording: the policy often closed the gripper on the way in, and it pulled sideways so the plug came half out. Two narrow skills act through the station's bounded move. *Regrip*: if the gripper shut on nothing, open to 0.6, lower 1.5 cm and close; the match plan lowers 2.5 cm, after 0 of 6 regrips held at 1.5 cm. *Pull*: lift 6 cm and then 4 cm, 3 s per step, or one 10 cm lift over 4 s in the match plan, then hand back to the policy with "put the held cable down on the table to the right". An opt-in station hook adds a 20 N upward force while the right gripper holds. *Lock*: at the first look that would pass a stage, the harness switches the policy to "open both grippers and move the arms up" before the confirming look, because on the station a "stack as high as possible" prompt built a tower and then took it apart.

4.4 Keeping the policy fed

Three changes target the serving loop (Section 7.1). The harness stopped pulling 2 MB debug-view frames, which had slowed calls to 2–6 s, and reads 35 KB frames instead. In simulation, asynchronous pacing with a 10-row execution horizon at 20 Hz requests a chunk every 0.5 s, and pipelining two sessions in turn lets a new request leave before the previous answer returns; one laptop match logged 5.5 requests/s over the whole match at a 0.61 s median round trip. A station-side option hedges each request across two sessions and keeps the first answer [14], with a cutoff and a cap on calls. Pipelining and hedging multiply policy usage, which matters on a metered API (Section 8).

4.5 How the design evolved

The team and several AI coding agents wrote the harness in parallel, on separate branches. Table 2 lists the lines that fed the final version. The last station plan on `main`, `bot_v1`, runs the base model with the gripper-assisted unplug loop, short stacking prompts with no arm named, a writer for one-ball prompts, the lock, the live guard, station-side hedged calls, and a view-only LiveKit observer for the integration points.

4.6 A decision model for tuning

The rules above are hand-set constants. To tune them, we wrote them as a model in which today's rules are a special case. With evidence features $\phi(t)$ (judge verdict times confidence, judge progress, gripper transitions, detector boxes at the goal, cable-end boxes), stage completion is a recursive log-odds score

$$\ell_t = \gamma\, \ell_{t-1} + \textstyle\sum_i w_i\, \phi_i(t), \qquad \text{done} \iff \ell_t \geq \theta_{\text{done}}(s), \tag{1}$$

and the current rule is $\gamma = 1$ on agreement and 0 otherwise, one unit weight on the judge and $\theta_{\text{done}} = 2$. If each look has false-positive rate α and k independent looks are required, $P(\text{false done}) \approx \alpha^k$; θ_{done} is set per pass bar to keep this

Table 3: Classifiers and estimators in the harness. Latency is per decision on the development laptop unless noted.

Component	Decides	Model	Latency	Evidence
VLM judge	stage done, progress, holding, next-slot hints	Haiku 5.5, Gemini 3.1 Flash-Lite, GPT-6.1 Terra, Sonnet 5.5 (confirm)	2–9 s	60-decision benchmark, 25 configurations
Colour + shape detector	balls, cubes, sticks, hole direction	HSV thresholds, blobs, shape tests; no learned weights	76 ms (two wrists)	hand labels on 26 station frames
Live guard	illegal ball in a gripper	detector on wrist frames, 3 frames in a row	10 Hz; stop in 0.3–0.5 s	0 trips on 2,145 station wrist frames
Cable-end classifier	USB-A / USB-C end / neither	orange-blob proposals + MobileNetV3-small, 96 px, ONNX	12 ms per frame	137 held-out wrist crops, 25 runs
Gripper-width rules	holding, closed on nothing, regrip	thresholds on `/arms` widths	<1 ms	widths from 101 station rollouts
Joint fault nets	joint in fault now / within 0.25 s	TCN or transformer encoder, 16-step windows	0.8 ms (12 joints)	injected faults; 31 real rollouts
Start-pose gate	arm far from any recorded start	distance to teleop starts per prompt	<1 ms	recorded starts
USB port finder	free port, seated plug	strip template fit + patch CNN	–	planned, not built

under a budget. An attempt ends when a logistic fail hazard over failures, progress regression, stall time and elapsed time crosses a threshold. The next action maximises

$$Q(a \mid x) = P_a(\text{success within } c_a)\, V_s + W(L - c_a) - \lambda R_a, \tag{2}$$

where V_s is the stage's points plus what it unlocks, c_a the action's time cost, R_a its disqualification or illegal-ball risk, and $W(L)$ the expected points of the remaining stages with L seconds left, computed backwards from each stage's success-versus-time curve F_s. Continuing on stage s pays while $\frac{f_s(\tau)}{1-F_s(\tau)} V_s \geq \frac{dW}{dL}$, which gives each stage a timeout tied to its own hazard and to the time left. The implementation runs in shadow mode beside the runner and reproduces its decisions exactly on 5,000 random look sequences. Stage curves fitted by censored maximum likelihood to the simulator's runs give the unplug a 0.64 ceiling, no success before about 23 s, and half of that ceiling by about 96 s. We did not let the model decide during the event; the curves need station data first.

5 Perception and monitoring

Table 3 lists every classifier and estimator in the harness. They differ by four orders of magnitude in latency, and the design places each by that latency and by how much it can be trusted: slow, general judgements gate progress, while fast, narrow ones guard safety and veto the slow ones.

5.1 Vision-language judge

The judge receives a stitched grid of the stage-start frame, recent overhead frames and the newest wrist frames, with the local detector's boxes drawn on them, and answers JSON: `done`, `progress`, `status`, `holding`, `guard`, and optionally a hint for the slot filler. Two or three calls are kept in flight, staggered, which on the station halved the time between verdicts from about 9.7 s to 4.9 s. Confirmation follows two rules learned on the station. A fast judge (Gemini 3.1 Flash-Lite, about 2.6 s) missed a real unplug and rewrote prompts 11 times in one run, so every unplug pass is confirmed by Sonnet 5.5. A fast judge also missed a four-cube tower, so Sonnet re-checks the tower every 10 s on its own. Judge accuracy and latency are compared in Section 7.3.

5.2 Colour and shape detector

The detector answers in milliseconds where the judge takes seconds. It thresholds HSV per colour from a table file, extracts connected blobs, and classifies them by shape: compact blobs of a ball colour with elongation at most 2.1 and

roundness at least 0.28 are balls; cube-coloured blobs are cubes, split at a dip in their distance ridge when two touch; thin blobs are lying sticks; a ball-coloured blob centred on a cube is a stick standing in its hole; and a white blob larger than 0.6% of the image is the bin, never a ball. It also reports which way a cube's hole runs. Boxes are drawn on the judge's frames, and the detector feeds both guards. It never touches the policy's inputs or prompts. The first version was tuned on phone photographs; on 8 Nov we retuned it on the station's own frames (Section 7.4).

5.3 Live illegal-ball guard

A thread reads both wrist frames at 10 Hz in parallel, with a 0.25 s timeout so a dead camera does not slow the live one. Three positive frames in a row, counted per camera and only on new frames, send `/stop` directly, 0.3–0.5 s after the ball enters view. A frozen camera therefore reads as blind, and the run stops after `max_blind_s`, rather than as all-clear. The runner re-checks for a trip around every command it sends, and refuses `/home` and `/reset`, which cannot be cancelled once begun, while the guard is armed.

5.4 Cable-end classifier

To tell the judge which cable end is where, a two-step classifier finds compact orange blobs (the anodised plug shells) and labels a 96 px crop around each with MobileNetV3-small: USB-C end, USB-A end, or neither. The model is a 6 MB ONNX file run by OpenCV, so the match laptop needs no deep-learning runtime. The first model (7 Nov) was trained on 891 hand-labelled crops from teleoperation videos and table photographs, split by source video. Blender renders of the cable with random pose, lighting and backgrounds did not help: synthetic plus real reached 92% accuracy against 94% for real only, and did worse on photographs, so the shipped model is real-only. On 8 Nov we hand-labelled 700 wrist crops from 89 station runs (478 usable) and retrained (Section 7.4).

5.5 Gripper-width rules

Gripper width is the most reliable single signal of what a gripper holds. From the station's 20 Hz joint logs, jaws stop at 4.5–14 mm on a plug body, about 3 mm on a wire or nothing, about 48 mm on a ball and 25–55 mm on a cube. The harness uses these in three ways: a gripper "closed fully on nothing" overrides any judge claim that it holds something; a closure on nothing at the plug triggers the regrip skill; and a gripper stopped on an object blocks re-prompting.

5.6 Joint fault detection

The arms have no brakes, so we built a per-joint fault monitor. Each joint's last 16 readings (0.8 s at 20 Hz) of eight features go in: position in range, margin to the nearer limit, velocity toward it, tracking error, commanded velocity toward the limit, and measured, commanded and differential torque. Two flags come out: in fault now, and in fault within 0.25 s. Training data is real policy motion with injected faults (limit runs, blocked joints, 1.5–6 Hz oscillation, overload, overspeed) and near misses, with labels taken from the resulting signals. We first trained on 413k samples from simulated matches and searched 15 architectures; a dilated causal TCN [15] (32 channels, 4 layers, 10.5k weights) won. We then retrained without torque, which the station log lacks, on the station's own rollouts, where a transformer encoder did better (Section 7.4). A rule-based observer runs alongside, because short overspeed bursts are better caught by rules.

5.7 Start-pose gate and the planned port finder

A VLA is sensitive to its starting configuration, and most demonstrations did not start from home (Section 7.6). Before each stage the harness logs the distance from the current joints to the starts recorded for that prompt and, on stages marked `home_before`, homes once if any joint is more than 0.35 rad from every recorded start. A USB port finder for the plug-in stage is designed but not built: a template fit to the strip's four rigid ports gives port indices that survive partial occlusion, and a small patch classifier decides empty, occupied or occluded, with a seated check that requires the judge and three consecutive "occupied" frames.

6 Simulators

Offline stress simulator. The real runner and judge client run unchanged against a scripted world. A virtual clock advances on every harness sleep, so a judge call returns only after its simulated latency and races between checks, verdicts and the policy happen as they would live. A table model tracks cables, gripper contents, the tower and the bin; each prompt has a try time and a success probability, with switches for failure modes seen on the station (the policy keeps the plug, picks the illegal ball, brushes a block, knocks the tower). Faults use the exact error types from the station log, plus frozen, empty, corrupt and lagged frames and judge outages. A scripted judge answers from the table state at the newest frame's capture time, and a referee scores rubric v4. A match takes about 5 s, so 37 scenarios × 5 seeds run on a laptop.

MuJoCo table with the live policy. A drop-in replacement for the station agent serves the same HTTP routes on Dream's MuJoCo YAM example [16] with a relay table, using Dream's own control loop and the live Dream-1 API; the harness cannot tell it from the station. A referee and an oracle verifier read object poses. It runs in Docker on a four-node home cluster and on cloud batch VMs, with a probe call before each batch and matches marked invalid when infrastructure, not the robot, fails.

Rebuilding the table. The first table (7 Nov) was a stand-in: a silver four-port hub, ten plain blocks, six white golf balls counted as legal, and a wire bin. It got the rules right and the scene wrong. On 8 Nov we rebuilt it from station photographs (Figure 1b,c): the strip at its catalogue size (25.7×9.7×3.0 cm) with raised outlet panels, switch, cord and a row of USB slots at the robot end; orange USB-A-to-USB-C cables with a seven-link cord and a seating check per port type; three 6 cm cubes with 2.8 cm through holes, a 2×2×10 cm square stick and a 2.5×11 cm round stick; a slotted white basket; green, yellow and orange legal balls and a white illegal one; and a light laminate table tuned so the white ball stays detectable. Software rendering was too slow to keep the simulation real-time on CPU-only VMs: three 4096-px shadow maps and 4× antialiasing cost about 320 ms per three-camera capture, so physics ran at 0.55× the match clock. Dropping shadows and antialiasing under software rendering brought a capture to about 50 ms. A later fix runs the harness on simulation time, so timeouts mean the same thing however fast the simulator runs. The rebuilt table is on the `cloud-sims` branch; the 8 Nov sim-time batches below branched from `main` and still used the first table.

7 Experiments

We ask six questions.

Q1 Where does a match's time go in the serving loop? (Section 7.1)
Q2 Does the harness survive the station's faults without losing matches? (Section 7.2)
Q3 Which judge is accurate and fast enough, and on which question? (Section 7.3)
Q4 How accurate are the local classifiers on station data? (Section 7.4)
Q5 What does the policy achieve under the harness, in simulation and on the station? (Section 7.5)
Q6 Why did the team's fine-tune do worse than the base model? (Section 7.6)

Data sources are kept apart: the station's inference log (52 runs, 1,650 calls), 101 scored station rollouts, the stress simulator (scripted policy), the MuJoCo simulator (live policy), and offline benches.

7.1 Q1: The serving loop

The median round trip was 1,116 ms (p10 800, p90 2,733, max 8,655; Figure 3). Model serving took 293 ms and the network 40 ms at the median, and the provider's gateway 993 ms. Each request carries about 1.6 MB of images, so uplink matters: at 8.8 Mbit/s up, calls took 4.7 s. Because the median sits on the pacer's 900 ms threshold, 67% of chunks (1,101 of 1,650) ran stop-and-wait, arriving every 3.54 s for 2.4 s of motion, against every 1.55 s with no gap in async mode. In stop-and-wait mode the arms therefore had motion queued for about two thirds of each cycle; we computed this from the log and did not watch the arms to confirm it. The harness adds 5–8 s per stage boundary (a poll, a judge call, a confirming look). For four or five stages we estimate 25–40 s of a match from those times. There were 16 policy errors in the window: 4 safety refusals (one triggered by a person's hand holding a plug), 4 `SVD did`

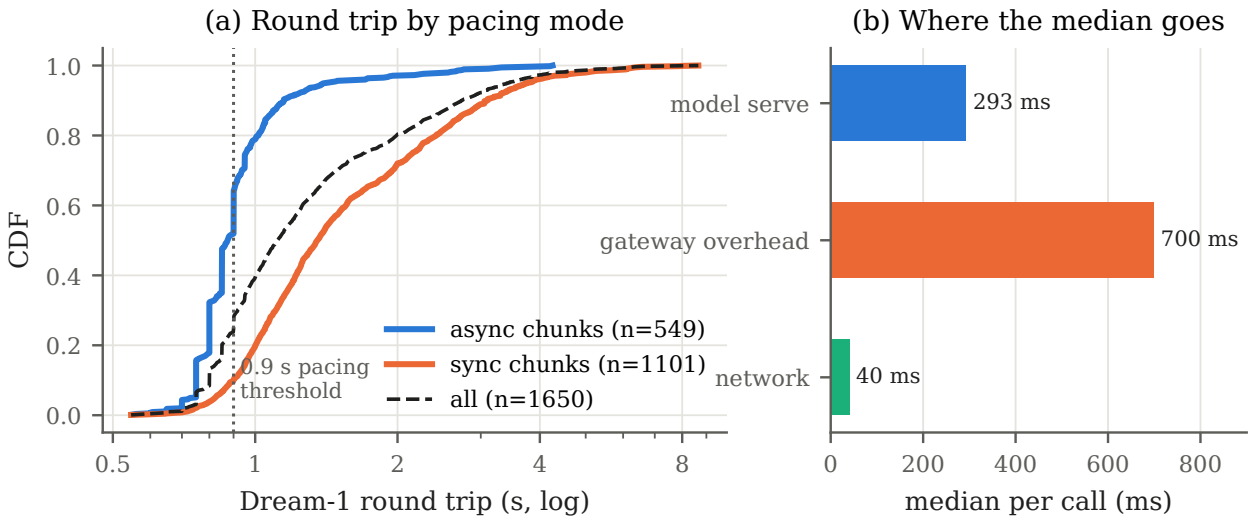


Figure 3: Dream-1 round trips on the station, 52 runs and 1,650 chunks between 18:06 on 7 Nov and 05:22 on 8 Nov (UTC). (a) Stop-and-wait (sync) chunks carry the slow tail; the pacer switches mode around a 0.9 s median. (b) Median components as logged; medians are not additive.

Table 4: Dream's Metal driver on a simulated Damiao bus (0.4 Hz sweep, gripper late half the time). Tracking RMS per joint, degrees.

Setting	Bus tick (Hz)	j0	j1	j2	j3	j4	j5
Stock: 30 Hz, FF τ 0.8 s	30.0	2.24	2.89	3.64	4.46	1.73	1.74
60 Hz requested	47–52	2.16	2.80	3.53	4.44	1.63	1.63
30 Hz, FF τ 0.15 s	29.8	1.37	1.73	2.19	2.31	1.18	1.19
60 Hz, FF τ 0.15 s	50.4	**1.27**	**1.60**	**2.03**	**2.27**	**1.05**	**1.05**

`not converge` errors in the client IK, 4 empty action buffers and 4 replaced sessions. The harness recovers from the first three without ending the match.

On the arm side, an offline bench of Dream's Metal driver on a simulated bus (Table 4) shows the feed-forward filter, not the bus rate, limits tracking: shortening its time constant from 0.8 s to 0.15 s cut joint RMS error by 40–50%.

7.2 Q2: Robustness in the stress simulator

Overnight fixes turned faults that used to end a match into small losses (Table 5). The plug case mattered most: when the policy kept holding an unplugged plug, the old harness's next prompt led it to plug the cable back in, which the rubric forbids, in 4 of 4 seeds; the fixed harness never did. Across whole configurations (Figure 4), judge speed is the main lever on points because a slow judge rarely leaves time for balls: at measured latency, configurations binned 0 to 0.09 balls per match. But a fast judge without the live guard reached the balls and lost 28 of 185 matches to the illegal ball, at a mean of 70.5 points. The final build with the guard scored 69.8 points (95% CI 65.3–74.4) and lost none to the ball. The two runs also differ in harness build, and the guard's detector in this simulator reads the simulated held object. In a separate six-seed test with frames 2 s old, the harness lost 4 of 6 matches to the illegal ball without the local detector and 2 of 6 with it. The final hybrid plan's 4 remaining losses are disqualifications in a scenario where the policy brushes a block during the unplug, which no supervisor can undo. In a separate sweep of 24 scenarios and 4 seeds, removing synchronous judge checks from stages the sensors can answer raised mean points from 54.9 to 60.0 with no new losses. These scores test harness logic against a scripted world whose unplug probability (0.8 per try) is far above the real policy's.

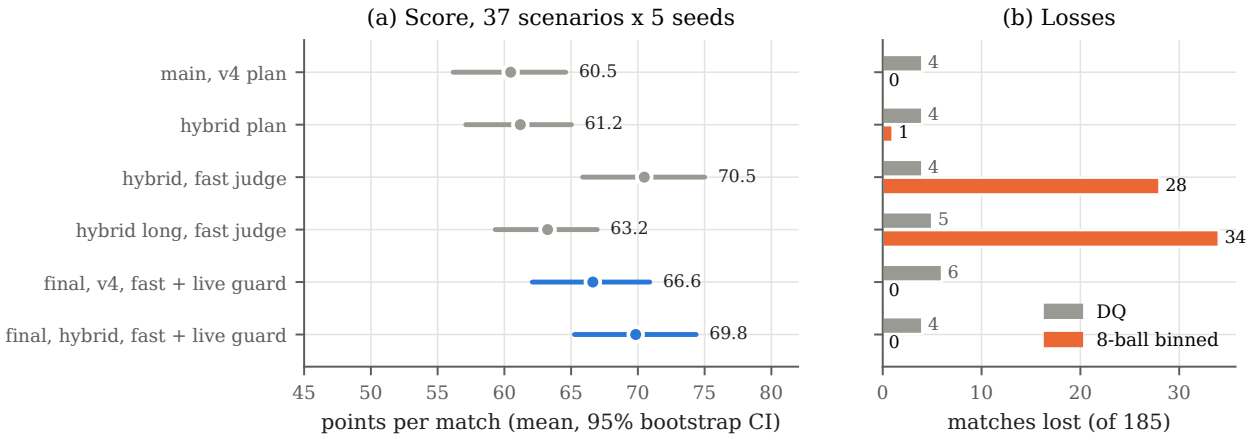


Figure 4: Stress simulator, 185 matches per configuration (37 scenarios × 5 seeds). "Fast" judge: 4–6 s per call; otherwise 6.5–13 s, taken from our Haiku 5.5 latency check. Both ranges are inputs. Blue: the final build with the live guard. Per-scenario means are in Appendix A.

Table 5: Points per seed in the stress simulator before and after the robustness fixes (4 seeds).

Scenario	Before	After
Happy path	87, 74, 90, 116	83, 94, 95, 95
Policy refuses on first start	0, 0, 0, 0 (match ended)	83, 94, 95, 95
Empty action buffer mid-run	10, 10, 15, 15 (ended)	79, 94, 95, 100
IK SVD failure	10, 0, 10, 10 (ended)	83, 84, 95, 95
12 s of corrupt frames	20, 15, 20, 20	83–90
Policy keeps the unplugged plug	plugged it back in, 4/4	never
Unplug brushes a block	DQ 4/4	DQ 4/4

7.3 Q3: Judges

Stage-done questions were easy for strong models; the guard question ("is a gripper holding the illegal ball?") separated them (Figure 5). GPT-6.1 Terra reached 98% done and 97% guard accuracy at a 2.8 s median with no reasoning, and 100% on both at 3.7 s with low reasoning, for about $0.07 per 60-decision match. Haiku 5.5 was perfect on done, but through OpenRouter it caught the held ball in 3 of 10 looks with no reasoning and 8 of 10 with low reasoning. Sonnet 5.5 was perfect on both at 7.1 s through the CLI, which includes launching the process. On a separate 120-decision run, Haiku 5.5 caught a held illegal ball 55% of the time. The 30 guard questions per configuration are ten looks at one simulated image of a held 8-ball and twenty looks at frames with none, and every configuration we report answered the twenty correctly. The ranking is therefore coarse. Two conclusions held on the station as well: no judge stands alone on safety, and a fast judge needs a slower one to confirm pass bars.

7.4 Q4: Local classifiers on station data

Detector. Retuning on the station's frames fixed three errors the phone photographs had hidden (Table 6): a pale yellow-green ball fell between the yellow and green ranges; half-lit balls read as crescents and were dropped; and the white bin was boxed as a white ball in 1,286 of 2,010 frames, which would have fed false alarms to the judge. On held-out frames ball recall rose from 20/27 to 26/27 at 100% precision and block recall from 16/23 to 23/23. The wrist guard tripped on none of 2,145 station wrist frames before or after; none of them shows a held white ball, so this measures false alarms only. Balls inside the bin and a white ball pressed against the bin are still missed.

Cable ends. The first model called a USB-A end lying flat "USB-C" with 0.71–0.95 confidence and missed many USB-A ends. Adding 478 labelled wrist crops (115 USB-A, 82 USB-C, 281 neither; 222 unsure crops left out), split by run, raised USB-A recall from 0.68 to 0.84 and USB-C precision from 0.83 to 0.92 (Table 7). The shipped model

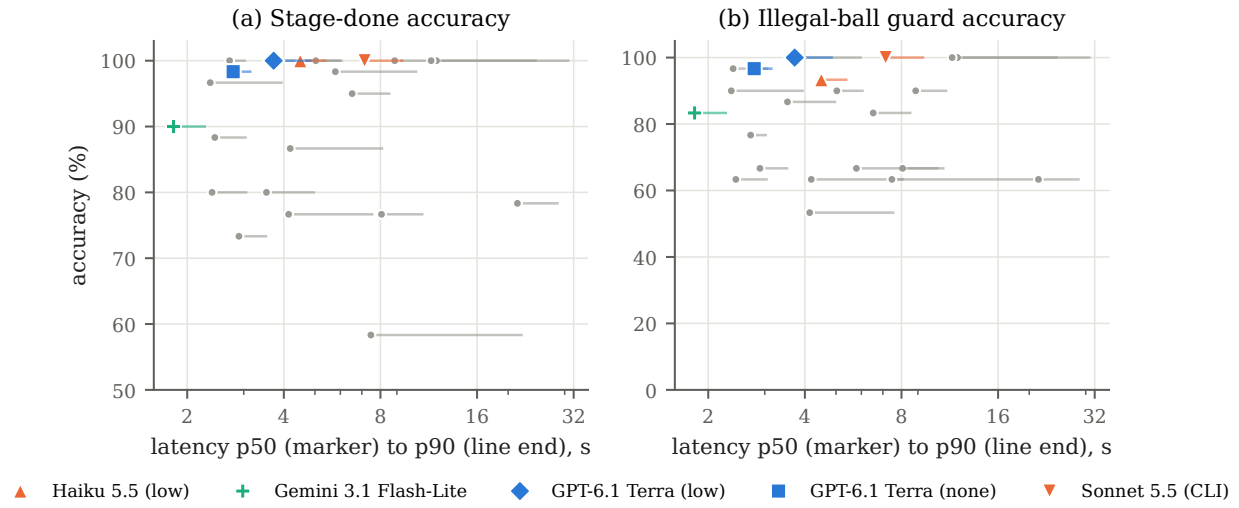


Figure 5: Judge benchmark: 25 model and reasoning-level configurations, 60 decisions each (30 guard questions) on the same frames, via OpenRouter or the Claude CLI. One configuration whose every call failed is omitted, leaving 24.

Table 6: Detector retuned on station overhead frames (hand labels). An object counts as found when a box of the right class covers its labelled centre.

	Tuning frames (14)		Held-out frames (12)	
	before	after	before	after
Balls found	24/46	42/46	20/27	**26/27**
Ball boxes that are balls	77%	100%	83%	**100%**
Blocks found	32/40	39/40	16/23	**23/23**
Block boxes that are blocks	86%	95%	80%	**92%**
All 2,010 overhead frames of 8 Nov: bin boxed as a white ball in 1,286 frames before, 0 after.				

is retrained on all 1,369 crops. A hand check of every box on one held-out run found about 62 of 64 correct, with one new false box on the strip's red switch. Neither model finds ends on the overhead camera, where a shell is a few pixels wide.

Joint faults. On simulated motion with torque, the TCN reached 0.91 timely recall at 0.2 false alarms per joint-hour, against 0.50 and 197 for the rules, whose limit projection fires on every fast move toward a limit (Table 8). Without torque, on the station's rollouts, the TCN lost most of its recall on blocked joints (0.20), and transformer encoders did better (0.78 for d32×2). Larger encoders bought about 0.05 for 4–7 times the weights. No model alarmed on the 0.83 untouched test joint-hours, so that column cannot rank them. The rule observer raised about 100 alarms per joint-hour on real runs, because two joints rest at their soft limit at home and every return home projects past it. Faults in all of these sets are injected; none of the models has seen a real fault.

7.5 Q5: The policy under the harness

Station rollouts. Daniel Carrera scored 101 station rollouts (171 prompt steps, 8 Nov) against before-and-after overhead frames and the 20 Hz joint log. Table 9 summarises the families. The clearest result is to name the goal, not the motion: the policy carries out the first verb of a chained prompt and dithers on the rest. Coached prompts did worst; a 574-character cable prompt unplugged nothing in 286 s. For cables, the best approach prompt ended with the jaws on the plug body in 15 of 26 steps, and with the harness pull, 8 of 12 pulls still held the plug after a 10 cm lift, meaning the plug was out. Balls worked when one named ball lay near the right arm (2 of 5 in the bin; the misses were a ball in mid-table). The half-open approach prompt came from a session in which a Claude Opus model operated the station directly: between 11:10 and 13:04 on 8 Nov it sent prompts and scripted pulls by hand, in 15–30 s steps, outside

Table 7: Cable-end classifier on 25 held-out station runs (137 wrist crops) at the 0.7 confidence threshold.

Training data	USB-C prec.	USB-C rec.	USB-A prec.	USB-A rec.	Accuracy
7 Nov: 891 video and photo crops	0.83	0.91	1.00	0.68	0.91
+ 478 station wrist crops	**0.92**	**1.00**	0.95	**0.84**	**0.96**

Table 8: Joint fault detectors. Top: simulated motion with torque, held-out match file, threshold for ≤1 false alarm per joint-hour. Bottom: station rollouts without torque (train 21 runs, validation 5, untouched test 5; one seed).

Model	Weights	Timely recall	Blocked	Overspeed	False alarms / joint-h
Simulated motion, with torque (test file)					
TCN 32×4	10,530	0.91	0.94	0.68	0.2
Rule observer	–	0.50	0.01	0.93	197
Station rollouts, torque-free (validation)					
TCN 32×4	10,530	0.75	0.20	1.00	1.9
Encoder d32×1	9,474	0.86	0.67	0.90	1.7
Encoder d32×2	18,018	0.87	0.78	0.84	1.2
Encoder d64×4	135,746	0.92	0.85	0.90	1.9

the harness. The policy itself rarely pulled. It lifted a plug 4 times all day, and the other cables came out through the scripted pull. The rubric's own wording, "stack the blocks and sticks as high as possible", stacked in 2 of 3 runs. The fine-tune made no cable grip in about 10 minutes.

MuJoCo, first table. Base Dream-1 through the harness unplugged a cable in 14 of 25 matches (first unplug median 57 s, range 28–269 s for the stage), then kept holding the plug: our notes count 0 of 54 put-down tries. Nine matches reached a stacking stage, and all 23 stacking tries in them failed. Three matches were disqualified, two for touching a block before a cable was out and one for touching a ball before the tower passed. In a separate 12-trial two-arm test, naming an arm in the prompt succeeded 0 of 4 times, and the other arm still moved for 10–19 s. On simulation time, six matches of the template-engine plan scored 42, 42, 41, 15, 15 and 15 points (mean 28.3), passed the tower bar in 3 of 6, and binned no balls. A relay-order batch got the first unplug in 12 of 12 matches and a second in 8 of 12, seated no plug, then crashed at the first stacking stage on a slot-filler bug, now fixed with a fallback and a test. These two sim-time batches are recorded in our notes only; their logs were not kept.

MuJoCo, cloud round 1. A comparison of async pacing, operator notes and judge choice (about 28 concurrent sessions) ran while the table was being rebuilt, and its saved frames show both versions. It stopped when the account's inference credits ran out: the service reports this as an unauthorised, disabled key while account calls keep working. Only 10 of 29 matches were valid, scoring 0–42 points, too few to compare the configurations. A rerun of the relay batch hit the same limit 34–99 s into each match. We now probe the policy with one call before every batch.

7.6 Q6: Why the fine-tune aimed high

Collection paths. Demonstrations came from two paths on the Metal arms. In the leader-follower path one arm runs MakerMods' gravity-compensated leader mode ($k_p = 0$, Pinocchio [17] gravity and friction feed-forward) and the other follows; we added the leader's gravity model to the follower's frames, which brought shoulder-lift tracking from 4–6° low to +0.2°. In the Quest headset path we added heading calibration, a One Euro filter [18], a wrist-flip guard, a jump guard and gravity feed-forward. Replaying 40 fast-wrist events through the IK, swings over 90° fell from 7 to 2 with tool position held within 8 mm. On a separate leader-motion bench, wrist lag fell from 105 to 27 ms. These Quest fixes came after the fine-tune's data was recorded.

Diagnosis. The team fine-tuned Dream-1 on 170 episodes (48,413 frames), 1,000 steps, learning rate 10^{-5}, batch 32, end-effector actions. On the station its tool stayed 8–13 cm above the table while reaching and closed at 11–16 cm, where demonstrations grasp at 4–6 cm and the base model reached 0–6 cm the same day (Figure 7b). The cause is in



(a) (b) (c)

Figure 6: Station C on 7 Nov. (a) The target tower for the stacking stage: three cubes, a yellow stick and a pink stick. The background behind the table is replaced with flat grey to remove bystanders. (b) A gripper holding the pink stick, with the cables still seated in the strip. (c) The table late in the evening with an orange ball in the bin. All three are cropped to the table.

Table 9: Prompt families on the station, base Dream-1 unless noted (101 rollouts, 8 Nov). Outcomes from overhead frames; grips from gripper widths.

Task	Prompt (abridged)	Outcome
Unplug	"With the right gripper half open, move down onto the plug . . . close . . . " + harness pull	jaws on plug in 15 of 26 steps; 8 of 12 pulls held the plug
Unplug	"Remove the first cable on the right from the power strip . . . then place it on the right side"	one cable by the policy alone, then 0 in 4 repeats
Unplug	574-character coached prompt (approach angle, cable vertical, two at once)	0 cables in 286 s
Unplug	judge-written micro-rewrites ("regrip", "hover", "lower")	0 cables; 9 of 15 sent while holding the plug
Unplug (FT)	the fine-tune's trained task text	0 grips in about 10 min
Ball	"With the right arm, pick up the <colour> golf ball <where> and drop it into the white basket"	2 of 5 in; misses were mid-table
Ball	two arms in one prompt; left arm across the table	no grip
Stack	"stack the blocks and sticks as high as possible"	stacks in 2 of 3 runs
Stack	"put the pink cube on top of a blue cube"	most decisive grasps (6 of 6); one correct stack, one wrong cube

the data (Figure 7a). The 73 Quest episodes were recorded through the kit's follower, which sends zero torque, so each joint sagged. On slow frames their actions sit 3.7, 4.8 and 5.4 cm above the measured tool in the three Quest recording sets. The 97 leader episodes apply gravity feed-forward, as the live station does, and their offsets lie between −5 and +7 mm. The model learned two incompatible maps from state to action, and live, the Quest map makes the arm rise and be asked to rise again. We ruled out camera mapping, view swaps, rotation, colour, resolution, camera position, joint order, units, gripper scale, tool frame and schema. Only 9 of 69 unplug demonstrations start near the home pose every live run starts from, which the start-pose gate now logs. An open-loop replay on a fitted MuJoCo model of one Metal arm found that no gravity scaling alone reached grasp height; a right-arm joint offset of 1.5 times the Quest sag did, with 1.8 cm of margin. The simpler fix is in the data: relabel each Quest action as the measured joints three frames later and screen every folder for steady-state action-minus-state offsets above about 1 cm before training. A corrected 500-step fine-tune takes about five hours, so the station ran base Dream-1.

8 Discussion and limitations

What the evidence supports. The classifier results (Tables 6–8) and the latency analysis rest on station data. The prompt findings rest on 101 scored station rollouts, with small counts per family. The robustness and judge-latency conclusions rest on simulation and fixed-frame benchmarks.

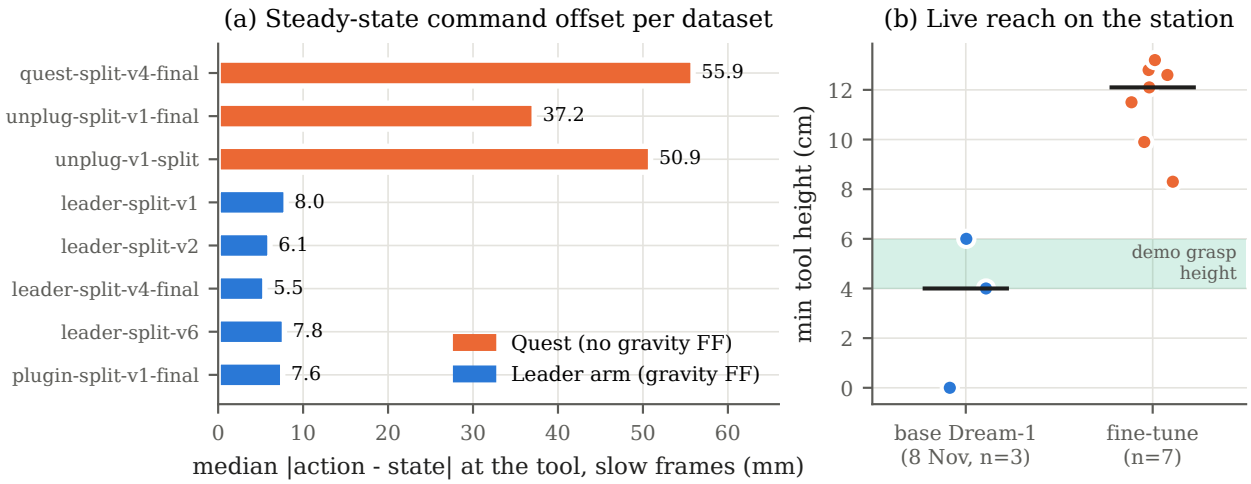


Figure 7: (a) Steady-state offset between commanded and measured tool position (frames with joint speed under 0.15 rad/s, so lag is excluded). Quest folders were recorded without gravity feed-forward. (b) Lowest tool height under the unplug prompt on the station; bars are medians. The fine-tune never reached the height at which demonstrations grasp.

Limitations.

- **No per-stage success rate from full harness matches on the station.** The station's inference log records calls and errors, not outcomes. The prompt study scores individual rollouts, not five-minute matches under the final harness.
- **Simulator fidelity.** The stress simulator's probabilities are assumptions. The MuJoCo arm is a YAM, not a Metal arm, so a Metal fine-tune cannot be evaluated there, and the rebuilt table has no clean comparison runs yet because of the credit limit.
- **The real judge was never run against the MuJoCo referee**, so false and late "done" rates on simulated frames are unmeasured.
- **Classifier test sets are small**: 12 held-out detector frames, 137 cable crops, five test rollouts for the joint models, and no frame of a held white ball for the guard.
- **Metered inference shaped the experiments.** Pipelining and hedging multiply policy calls, and two batches ended when credits ran out.
- **An independent audit** of the harness by a separate coding agent, before the matches, found 26 issues, two critical: a logging error in the stop path could leave the robot running, and the tower pass bar could pass on one image without the stability dwell. Several were fixed on later branches; the rest are listed in the repository.

Lessons. Under gating rules the supervisor's false positives are the main risk, so confirmation must use independent evidence. Safety checks should not share a path with slow reasoning. The serving loop should be measured before prompts are tuned; our own debug traffic was part of the latency problem. A VLA is best prompted by naming the goal and the object, with motions it cannot follow from words moved into small, separately tested skills. Classifiers tuned on photographs need retuning on the robot's own cameras. And action-minus-state should be checked per data source before any fine-tune.

9 Conclusion

For a hosted VLA under a strict rubric, most of the engineering a team can do sits outside the policy: when to declare a stage done, how fast to stop, what words to send, how to keep the serving loop busy, and whether the demonstrations agree with the robot's control path. We built a harness around those decisions with a layered set of classifiers placed by speed and trust, two simulators to test it, and the tools to find where the data went wrong. The next experiments are full matches on the station with per-stage logging, the real judge against the MuJoCo referee on the rebuilt table, and a fine-tune on gravity-corrected data compared with the base model on the same seeded scenes.

Acknowledgments

We thank MakerMods and Dream Labs for the stations, Dream-1 access and the event kit, and Founders Inc for hosting Physical AI Hacks. Much of the code and analysis was written with AI coding assistants (Anthropic's Claude Code and OpenAI's Codex) under the authors' direction; the authors chose the experiments, ran the station, and checked the results reported here.

References

[1] Anthony Brohan, Noah Brown, Justice Carbajal, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[2] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[3] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[4] Michael Ahn, Anthony Brohan, Noah Brown, et al. Do as I can, not as I say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.

[5] Wenlong Huang, Fei Xia, Ted Xiao, et al. Inner monologue: Embodied reasoning through planning with language models. In *Conference on Robot Learning (CoRL)*, 2022.

[6] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[7] Yuqing Du, Ksenia Konyushkova, Misha Denil, et al. Vision-language models as success detectors. *arXiv preprint arXiv:2303.07280*, 2023.

[8] Zeyi Liu, Arpit Bahety, and Shuran Song. REFLECT: Summarizing robot experiences for failure explanation and correction. In *Conference on Robot Learning (CoRL)*, 2023.

[9] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems (RSS)*, 2023.

[10] Philipp Wu, Yide Shentu, Zhongke Yi, Xingyu Lin, and Pieter Abbeel. GELLO: A general, low-cost, and intuitive teleoperation framework for robot manipulators. *arXiv preprint arXiv:2309.13037*, 2023.

[11] Remi Cadene, Simon Alibert, Alexander Soare, et al. LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch. https://github.com/huggingface/lerobot, 2024.

[12] Xuanlin Li, Kyle Hsu, Jiayuan Gu, et al. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024.

[13] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[14] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[15] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.

[16] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.

[17] Justin Carpentier, Guilhem Saurel, Gabriele Buondonno, Joseph Mirabel, Florent Lamiraux, Olivier Stasse, and Nicolas Mansard. The Pinocchio C++ library: A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives. In *IEEE/SICE International Symposium on System Integration (SII)*, 2019.

[18] Géry Casiez, Nicolas Roussel, and Daniel Vogel. 1€ filter: A simple speed-based low-pass filter for noisy input in interactive systems. In *ACM CHI Conference on Human Factors in Computing Systems*, 2012.

A Per-scenario stress results

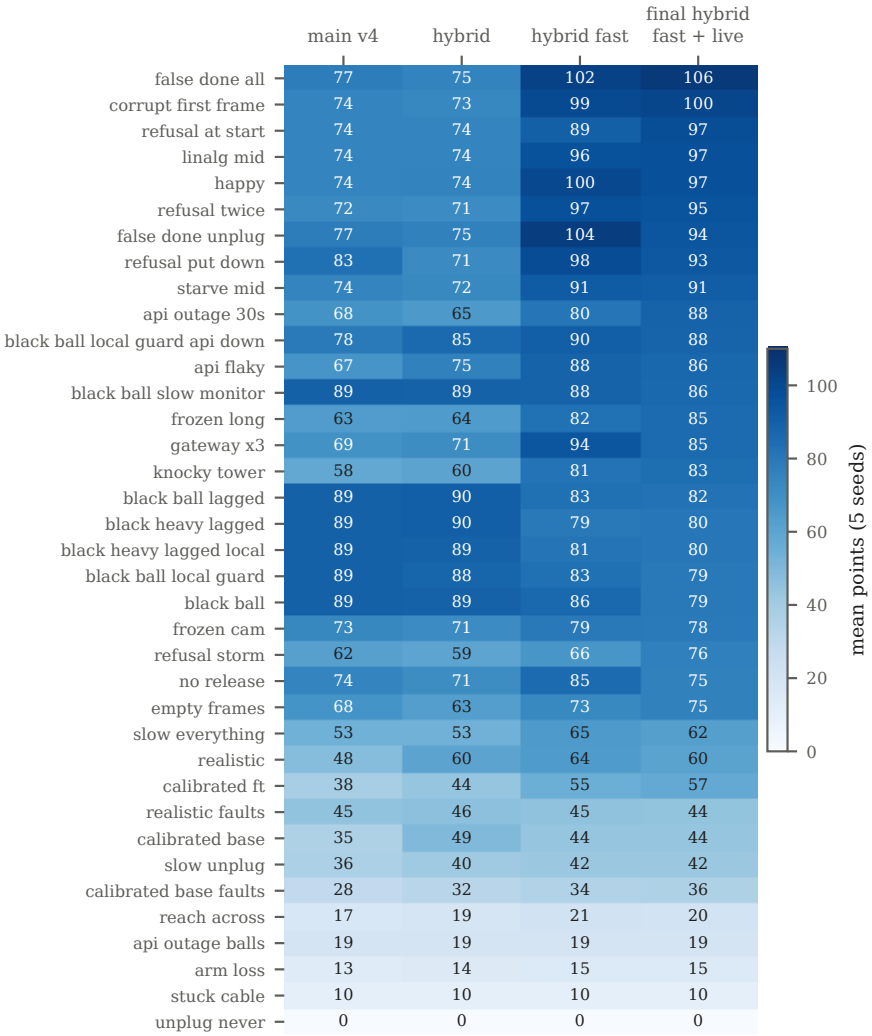


Figure 8: Mean points per stress scenario (5 seeds each), sorted by the final configuration. Scenarios include API outages, refusals, frozen and corrupt cameras, false "done" verdicts, an illegal ball near the bin, a knock-prone tower, and `unplug_never`, in which no configuration can score.

B Reproducing the numbers

- Stress sweep: `cluster/run_stress.sh 50 7 sweep1`, aggregated by `results/stress/final_20261108/analyze.py` in `task_simulation`.

- MuJoCo matches: `python sim_station.py -policy dream -scene hill_climb` with `experiments/sim_match.py`; needs a Dream-1 key with credit. The rebuilt table is `sim_scenes/hill_climb_table.py` on the hill-climb-harness `cloud-sims` branch.
- Figure 1b,c: `paper/render_sim_table.py` renders that table with MuJoCo, no policy and no network.
- Latency, stress and judge figures: `paper/make_figures.py` reads the logged files listed in its header.
- Classifier evaluations: `docs/CLASSIFIERS_8NOV.md` (branch `fix/classifiers-8nov`) and `docs/JOINT_FAULT_REAL.md` (branch `joint-fault-encoder`); station frames with people in view are kept outside the repositories.
- Prompt study: `docs/prompt_comparison_2026-11-08/` on branch `docs/prompt-comparison-2026-11-08`.
- Quest replay bench: `quest/bench` in `follower_code` (branch `quest-continuity`).